

星云训练营 - 实训作业 - 基础部分

1.背景

本次实训作业要求构建一个 Agent 基础框架，旨在帮助同学们深入理解 Agent 的工作原理，对 Agent 有更深入的认识，进而能够构建出更实用的 Agent。

同时，实训将大量使用 KSCC 进行 AI-Coding 工作，也借此让大家对 AI-Coding 有更深刻的理解，在新的研发范式下找到自己的位置。

2.agent 介绍

实训作业分为 Agent 基础框架与 Agent 业务赋能两部分。Agent 基础框架是根基，在后续业务赋能的实践过程中，也会反过来推动基础框架的持续完善。



3.模块

模块	基础链路（一句话）
01 登录用户权限管理	登录拿 sessionId → 每次校验合法性 → 凭工具权限 → 框架统一拦截权回灌 → 多用串
02 人机协同	任意节点能中断存状态 → 危险操作 yes/no 程内同 run ID 取回复原续跑 → Web 承载批 - 恢复
03 多 Agent 与工具调用	工具注册发现按名调用 → 单 Agent 自主闭环 + 兜底 → supervisor 中心路由分派汇总 Agent fresh context
04 上下文管理	按消息数裁 → 按 token 裁 + 估算 → 保 sy 不破坏 tool 配对 → 超阈值摘要压缩
05 记忆管理	短期按 thread_id 存会话快照 → 长期按 u 跨会话偏好 → 内存版先跑通 → 跨会话记忆

4.agent 框架

框架	语言	维护方	一句话定位
LangGraph	Python	LangChain (OSS)	基于状态图（Pregel/Beam 启发）的状态 Agent 编排运行时
LangGraph4j	Java（补充）	bsorrentino（社区单人）	LangGraph 的 Java 移植：状态图 + 循环 / 条件路由 + 检查点 + 原语级 interrupt/resume，补 LangChain4j 编排运行时短板
Eino	Go	字节跳动 / CloudWeGo	强类型组件抽象 + Graph/Chain/Workflow 编排 + ADK 能力体

5.各模块基础题目页

模块 01：登录用户权限管理（身份与权限子系统）基础题目页

题目背景

要为通用 Agent 框架构建身份与权限子系统，统一管理用户会话、角色以及工具级访问控制，以此界定“谁能使用这个 Agent、谁能调用哪个工具、多用户数据如何隔离”。

作为框架的入口子系统，每一次 Agent 调用都会携带“谁在调用”的身份上下文，后续模块（如人机协同中的审批人、记忆模块的用户归属、会话上下文的范围）都依赖于此。

现有框架大多仅提供“标识 + 隔离”原语，工具级 ACL（谁能调用哪个工具）需框架自行实现——这正是本模块的核心任务。

核心知识点涵盖：用户身份与会话（Session）、角色与权限（RBAC）、工具级访问控制、多用户隔离，以及身份上下文传递。

任务要求

1. 用户登录与会话创建

提供登录入口，校验凭证后创建会话（Session）并返回 sessionId。后续 Agent 调用需携带该 sessionId，框架据此识别调用者身份。未携带或标识错误的调用将被拒绝。

2. 会话校验

每次调用均校验 sessionId 合法性。若攻击者伪造 sessionId，框架在入口处校验，当存储中查无该会话时即判定非法，直接返回 401，请求不进入 Agent 执行。（会话过期 / 续期 TTL 属于进阶档。）

3. 角色与权限模型 RBAC（工具 ACL 的前置）

定义用户、角色与权限集，至少包含两种角色（如 admin 可调用全部工具，visitor 仅可调用部分工具），权限粒度达到工具级。用户与角色绑定，应能明确说明管理员与普通用户各自可调用的工具范围。

4. 工具级 ACL——框架统一拦截

Agent 调用工具前，框架根据当前用户角色统一校验：通过则放行，越权则拒绝并将原因回灌给 LLM。校验逻辑集中在框架拦截层，不得分散在各工具内部。反例：若在 delete_user 等工具函数内部自行编写校验，新增工具极易遗漏校验，这是核心扣分点。

5. 多用户隔离

不同用户的会话和记忆数据须严格隔离，不可串读，以 userId/thread_id 作为隔离键。当用户 A 与 B 同时发起“查我的订单”时，A 只能看到自己的订单，B 同理。身份上下文沿调用链透传，对 LLM 隐藏，对工具与节点可见。

6. Web 动态页面

需提供可现场运行的前后端页面，包含登录、用户与角色管理功能，并能直观演示权限差异：例如 admin 与 visitor 两个用户登录后询问同一问题，因权限不同，可能调用不同工具，或越权被拒并将提示回灌。

技术约束

1. Web 动态页面承载权限演示

前后端分离，web 动态页面

登录、用户 / 角色管理及权限差异的功能能演示。

2. 语言选择

在 Java、Python、Go 中仅选一种语言，自研本子系统所有内容。

3. 工具级 ACL 框架统一拦截

工具调用前的权限校验必须由框架统一拦截，禁止在每个工具内部手写校验。可以基于框架的拦截能力进行扩展。

4. 多用户隔离

会话与记忆数据按 `userId/thread_id` 隔离，切换用户不会串读数据。身份上下文沿调用链透传，对 LLM 隐藏，对工具与节点可见。

5. SessionStore 接口为必交付项

必须交付会话存储抽象接口 `SessionStore`，并至少提供内存版默认实现；持久化存储后端属于进阶档。

模块 02：人机协同（web 人机交互）基础题目页

题目背景

要为通用 Agent 框架构建“中断 - 恢复子系统”。自主执行的 Agent 通常会一气呵成地调用工具、完成动作，但在生产环境中，某些操作（如删除数据、发送邮件、转账、提交订单、调用计费 API 等）具有危险性、不可逆性 or 高代价，不能由 Agent 自行决断。人机协同（HITL）正是要解决这一问题：在 Agent 自主执行流程中设置暂停点与审批闸门：

- **暂停点**：Agent 运行到特定节点或工具调用前主动挂起，将“我打算执行 X，参数为 Y”呈现给人，等待裁决。
- **审批闸门**：人审阅后做出批准或拒绝的决策，Agent 拿到决策后从中断处恢复继续执行，无需从头重跑。

其背后还藏着一项工程硬要求：Agent 挂起后，进程可能重启或被跨进程调度，因此中断时的运行状态必须能够持久化保存，恢复时再准确还原。本模块的使命，正是把“任意节点挂起 + 状态持久化 + 随后恢复”抽象为框架的通用能力。

任务要求

1. Web 动态页面承载 HITL 交互

与 AI 的交互都是在 web 动态页面进行交互，包括中断展示与审批的人机交互，并可现场演示运行。

2. interrupt/resume 核心机制

框架提供在任意节点、任意工具调用处主动中断的统一入口。中断时携带待决策的 payload，恢复时传入人的决策，Agent 从中断点继续执行。

3. 内存版 CheckpointStore

交付 CheckpointStore 抽象接口（含 save/load/delete 等方法），并提供内存版默认实现。中断状态按运行 ID 或会话 ID 存取。

4. 批准 / 拒绝模式

演示最基础的人机协同——Agent 调用某个危险工具（如发送邮件、删除数据、转账）前中断，由人给出批准或拒绝的决策；批准则实际执行，拒绝则回退或切换执行路径。

5. 状态持久化（进程内恢复）

中断后，基于同一运行 ID，在进程内能恢复状态并继续执行。

技术约束

1. Web 动态页面承载 HITL 交互

前后端分离，使用 web 动态页面。可以演示。

2. 语言选择

在 Java、Python、Go 中仅选一门语言，自研本子系统全部内容。

3. CheckpointStore 接口为必交付项

必须交付状态持久化抽象接口 CheckpointStore（含 save/load/delete 等方法），并提供内存版默认实现。

4. 恢复语义须明确

文档需明确恢复时是“从中断行继续”还是“节点重跑”，并据此要求业务保证幂等性。

5. 身份与会话复用模块

中断状态按运行 / 会话 ID 存取，会话与身份信息直接沿用模块 1 的成果，避免重复建设。

模块 03：多 Agent 与工具调用（编排与工具子系统）基础题目页

题目背景

要为自研通用 Agent 框架构建编排与工具子系统——这是整个框架的执行核心。它需要回答两个根本问题：

1. 单个 Agent 如何自主决策调用工具？Agent 不应只是“一问一答”，而是一个“推理—行动—观察”的循环体（ReAct 循环：Reason → Act → Observe → 回到 Reason）。不会自主调用工具的 Agent，本质上只是一个带提示词的聊天机器人。
2. 多个 Agent 如何协作？本框架采用 supervisor 中心路由模式：中心 Agent 充当路由器，将任务分派给若干专业子 Agent（每个子 Agent 内部各自运行 ReAct 循环），最终汇总结果。

为何说它是框架的执行核心？身份权限（模块 1）决定“谁能用、谁能调哪个工具”，人机协同（模块 2）决定“危险操作前暂停”，上下文管理（模块 4）决定“对话变长不超窗口”——这些是约束与边界；而本模块，是真正让框架“跑起来”的执行引擎。

任务要求

1. Tool 抽象与 ToolRegistry

设计统一的 Tool 抽象：对外暴露工具元信息（名称、描述）及入参 schema（JSON Schema），对内执行业务逻辑，返回结构化的 ToolResult（包含 content / error / metadata）。实现 ToolRegistry 工具注册表，支持注册与查询，Agent 启动时从中获取工具集并喂给 LLM。内置若干示例工具（如天气查询、计算器、grep）。

2. ReActAgent：ReAct 循环与 MaxIterations 兜底

实现统一的 ReAct 循环引擎：Reason（调用模型决策）→ Act（模型发出 ToolCall）→ Observe（执行工具，将结果序列化为工具消息回灌对话历史）→ 回到 Reason，直至模型不再请求工具并给出最终答案。内置 MaxIterations 兜底（建议 10~20，可参考 Eino ADK 默认 20）：超限则终止循环并返回“已达迭代上限”的合理提示，杜绝死循环。所有工具 Agent 复用同一套 ReAct 逻辑。

3. supervisor 多 Agent：中心路由至 2~3 个子 Agent

实现 supervisor 中心路由模式：中心 Agent 充当路由器，根据用户问题决定调用哪个专业子 Agent（如数学 Agent、搜索 Agent、通用 Agent），各子 Agent 内部各自运行 ReAct，完成后将结果交回中心 Agent 汇总。supervisor 对外仍表现为一个统一执行入口的 Agent。

技术约束

1. 多 Agent 协作仅采用 supervisor 模式

使用 supervisor 模式作为多 agent 协作的方式。

2. Agent 内部执行统一采用 ReAct 模式

由 supervisor 调度的每个子 Agent，其内部循环必须为 ReAct。

3. 语言选择与框架对比

开发语言从 Java、Python、Go 中三选一，只实现一种语言。设计说明文档须包含对三种主流 Agent 框架（LangGraph、LangGraph4j、Eino）的对比分析。

模块 04：上下文管理（上下文窗口子系统）基础题目页

题目背景

在自研的通用 Agent 框架中，会话已能正常运转，也能在危险操作前中断并等待人工审批。然而，随着对话轮次不断累积，一个看似不起眼的问题会逐渐拖垮框架——消息历史持续膨胀。

LLM 的上下文窗口有 Token 上限，但每一轮都必须将历史消息全量输入。

这导致三个严重问题：

- ① 超窗口：消息总量超出上下文窗口，请求被提供商直接拒绝并报错；
- ② 成本爆炸：Token 用量随轮次近似线性增长，API 费用急剧攀升；
- ③ 延迟升高：输入 Token 越多，首 Token 延迟越高。

本模块要解决的核心问题是：在当前会话中，喂给模型的上下文究竟该保留什么、丢弃什么、如何压缩——构建一个对业务透明的中间层，自动将过长的历史裁剪或摘要至窗口限制之内。

任务要求

1. 按消息数裁剪

实现“保留最近 N 条消息”的滑动窗口裁剪策略，需确保 system 消息永久保留，且 tool 结果与 tool_call 消息保持配对不被拆散（原 04-3 已裁减并入此项要求，不单独计分）。

2. 按 Token 数裁剪与 TokenCounter

实现“保留最近 N 个 token”的滑动窗口裁剪策略，并提供 Token 计数能力，支持单条及多条消息的 Token 数量估算。

3. 摘要压缩

当上下文占用超过阈值（如窗口的 80%）时，将旧消息压缩为一段摘要后再裁剪，保留要点、丢弃细节，摘要触发条件可配置。

4. 长对话演示

在多轮持续对话中，当上下文超出窗口限制时，自动裁剪旧消息，保证不超窗口、不报错。

技术约束

1. 语言选择

从 Java、Python、Go 中三选一，仅用一门语言实现，但设计文档须包含对三个框架的对比分析。

2. 摘要压缩

使用 LLM 进行摘要压缩。

模块 05：记忆管理（记忆子系统）基础题目页

题目背景

前几个模块已搭建出一个能登录、能中断恢复、能管理上下文、能多 Agent 协作的 Agent 框架，但它仍有一个致命短板：记不住历史。

- 单次会话中聊到一半，一旦进程重启或换台机器，Agent 就会把刚刚说过的话全部忘掉——它缺少状态快照。
- 用户多次对话，每次新开会话都要从零开始自我介绍——它跨会话失忆，记不住用户的偏好、身份和过往操作。
- 多用户共用一个框架时，A 的记忆可能泄露给 B——它无法区分不同用户的记忆归属。

记忆分为两层，性质截然不同，不可混淆：

- **短期记忆（Short-Term Memory）**——会话内的状态检查点。以“线程 / 会话”为粒度，保存 Agent 运行至某一时刻的完整状态（消息历史、中间变量、待执行节点等）。它让单次会话可中断、可恢复（与模块 2 配合），并能从崩溃中续跑，生命周期与会话绑定。

- **长期记忆 (Long-Term Memory)** ——跨会话持久化的用户画像、事实与规则。以“用户/命名空间”为粒度，在多次会话中持续沉淀对用户的认知。它让 Agent 具备“换个会话仍记得你”的个性化能力，生命周期远超单次会话。

任务要求

1. 短期记忆

实现 CheckpointStore 的 save/load 方法，按 thread_id 隔离。同一会话内的多次调用可续接状态，不同会话之间互不干扰。

2. 长期记忆

实现 MemoryStore 的 put/get 方法，按 userId 存取用户画像与偏好事实（例如“用户喜欢简洁回答”“用户是 Java 开发者”）。

3. 内存版存储后端

提供 InMemoryCheckpointStore 与 InMemoryMemoryStore 两种默认实现，无需任何外部依赖即可运行。

4. 最小演示

跨会话用户偏好记忆——会话 A 中用户告知偏好并写入长期记忆，新建会话 B（不同 thread_id）可读取该偏好并据此回答。典型示例：会话 A 中用户说“我喜欢用 Python”，会话 B 中要求“帮我写个脚本”时，Agent 默认使用 Python。

技术约束

1. 语言选择

Java、Python、Go 中仅选一门实现。

2. 短期与长期分层

CheckpointStore（状态检查点）与 MemoryStore（事实条目）必须定义为两套独立抽象，不可混用。二者数据模型与生命周期截然不同：短期记忆是状态快照，长期记忆是事实条目。

3. 存储后端可替换

CheckpointStore 与 MemoryStore 均通过接口抽象，内存版与持久化版可平滑切换，业务代码无需改动。接口定义是必交付项，即使最终仅运行内存版，也必须明确给出接口。

4. **userId 作为长期记忆命名空间**

thread_id（短期隔离键）与 userId（长期命名空间前缀）均取自模块 1 的 AuthContext，从源头保障多用户隔离。此为隐私与安全红线。

6.QA